Simple PHP Server

- Runs a web server locally on your machine
- Uses built-in PHP web server
- Prints "Hello World" when accessed

PHP Script Example

```php
<?php
// Simple script to print Hello World
echo 'Hello World';
?>
```

- PHP programs are enclosed by `<?php ... ?>`.

CLI vs Server: Two Ways to Run PHP

PHP code can run two different ways.

- The right column shows the server way that this course always uses.
- The left column shows the script way to use PHP as the script language such as Python or Node.

	`php index.php`	`php -S localhost:8000`
What it is	Run PHP as a CLI script (like Python or Node)	Run PHP's built-in web server
Who sees the output	Only your terminal	A browser, over HTTP
Used in this course?	No — reference only	**Yes, this is how we run PHP**

- With the server, PHP code inside `<?php ... ?>` runs **on the server, not the browser**.
- It executes that code and sends only the **output** (HTML/text) back to the browser.
- The browser never sees the PHP source, only the final rendered page.
- PHP lets HTML pages include **dynamic content** (e.g., user/session data).

IMPORTANT

1. This was a revolutionary idea that made PHP successful.
2. Before PHP, a web server simply sends static web pages and a web browser renders them.
3. After PHP, a web server can **dynamically** generate web pages.
4. JavaScript runs in the browser to make pages **interactive** on the client side.

For reference — you will not use this in the course, but it helps to see the difference:

```
php index.php
```

- This reads and executes `index.php` directly.
- It is the same idea as running a Python script with the Python interpreter: no server, no browser, just a script.

Starting PHP Server

A project folder is a directory that holds all files for one web application.

- For this example, the folder contains `index.php`.
- Later, it can also contain CSS, JavaScript, images, and other PHP files.
- Keep related files together in one project folder.

Create a Project Folder

In Terminal, create a folder named `hello-php` and move into it:

```
mkdir hello-php
cd hello-php
```

Use VSCode

1. In VS Code, select **File > Open Folder...** and choose the `hello-php` folder.
2. In the Explorer panel, select **New File** and name it `index.php`.
3. Paste the Hello World PHP code into `index.php` and save it.
4. Select **Terminal > New Terminal** to open VS Code's integrated command line.

index.php

```php
<?php
// Simple PHP server script to print "Hello World"
echo 'Hello World';
?>
```

Run the Server

In VS Code's integrated Terminal, make sure you are in the `hello-php` folder.

Run:

```
php -S localhost:8000
```

If port 8000 is busy, use a different port number.

Accessing the server

- Access http://localhost:8000 via browser
    - You should see "Hello World" in your browser.

- Access via curl (command line tool)
    - You can also access your PHP server using the curl command, which is helpful for testing from the terminal or scripting API requests.
    - The response (e.g., "Hello World") will be printed directly in your terminal.

```
curl http://localhost:8000
```

IMPORTANT

You made a web server!

You are running a web server on your local machine.

- It receives HTTP requests and sends HTTP responses.
- A browser or `curl` uses the same HTTP protocol for a local or remote server.

Local vs. Public Server

- `localhost` is reachable only from your own machine.
- On a VPS, use `php -S 0.0.0.0:8000` and open the firewall port for public access.
- PHP's built-in server is for development only; Production deployments use a web server such as NGINX.

OPTIONAL

Optional: VS Code Extension

- Install "PHP Server" extension (Use **brapifra.phpserver** in the Extension search).
- Open your project folder and choose your PHP file (`index.php`) in the project folder.
- Use the extension:
  - Click the PHP Server button in the editor's top-right corner.
  - To stop the PHP server, use the **PHP Server: Stop project** command from the Command Palette.

Debugging a PHP program

- The simplest way to debug a PHP program is the `error_log()` function — it writes values to a log file instead of the browser.
- For more powerful tools (PHPStorm, Xdebug), see `4. Debugging_Optional.md`.

Use error_log() function

- Use `error_log()` to write debugging information to a log file.
- In this example, the `$data` array is written to `debug.log` in the same folder as this PHP file.
    - The `3` tells `error_log()` to append the message to the file path that follows.

```php
$data = ['foo' => 'bar', 'baz' => 42];
error_log(print_r($data, true), 3, __DIR__ . '/debug.log');
```

What You Did

1. You made a web application that prints out "Hello, World" using PHP.
2. You ran your web application locally on your computer using a PHP web server.
3. You accessed the web application locally via browser.